*codeblock

__/__/____

ARRAY

Q. How to find highest number in ~~abc~~ array

```
# Include <iostream>
using namespace std;
int main ()
{
int a[5] = {35, 70, 20, 10, 15};
int max, i;
int find_max (int a[], int n);
n = size of (a) / sizeof (int);
max = find_max (a, n);
Cout << 'Maximum No is = " << Max;

int ~~main~~ find_max (int a[], int n)
{ int max;
max a[0];
for (i=0; i<n; i++)
{ if (a[i] > max)
{
max = a[i];
}
} return max;
}
```

Q. Find for lowest number

__/__/____

Q.1 Inserting element in array

```
# Include <iostream>
  using namespace std;
  int main ()
 { int loc , i, val, n ;
  int a[n] ;
   cout << " Enter no. of elements";
   Cin >> n ; a[n];
   Cout << " Enter location to enter element :" ;
    Cin >> loc ;
   Cout << Enter value : ";
    Cin >> val;
   for (i = 0 ; i<n ; i++) for (i=n-1 ; i >= loc ; i--)
    {
    For (i=n-1 ; i >= loc ; i--)
     { a [i+1] = a[i]
      }
     a [i+1] = val ;
     for (i=0 ; i <= n ; i++)  → Display array
    { count <<a [i] ; }
  }
```

Q.2 Write code to display elements of array.

Q.3 Write code for deletion of element from array

```
↑ Cout << " enter array elements";
   For (i=0 ; i<n ; i++)
   { Cin >> a[i]; }
```

__/__/____

Addition :-

```
# include <iostream>
# define MAX 10
using namespace std;
int main ()
{ int a [MAX] , b [MAX] , sum [MAX] , n ;
  Cout << " Enter the length of array " ;
     Cin >> n;
    Cout << " Enter array 1 " << " \n" ;
    for (int i=0 ; i<n ; i++)
     { Cin >> a [i] ;
     }
     Cout << " Enter array 2 " << " \n" ;
     for (int i=0 ; i<n ; i++ )
      { Cin >> b [i] ;
      }
      for (int i = 0; i <n ; i++)
       { sum [i] = a[i] + b [i] ; }
       for (int i = 0 ; i <n ; i++)
       { cout << "\n" << sum [i] << " \t" ; }
         return 0 ;
       }
```

__/__/____

→ vectors in cpp is resizable array

→ Both vectors and array's are linear ds used to store multiple elements of c datatype

→ The difference between an array and vectors is that the size of an array cannot be modified at the execution time

→ A vector can grow or shrink at the execution time.

→ A vectors are implemented in header file name as <vector> so we need to include that header file in our program also the vectors are known as STL (standard template library) is a collection of template classes and functions, it provides preme components for common data structure and algorithms for promoting reusability efficiency and maintainability

→ The syntax to declare vectors are :-

- vector <datatype> name ;

ex:- vector <int> a ;

vector <int> a = {10, 15, 20}

vector <int> a (3, 5);

size of vec. ↵ ↳ initialization value

→ To print vector in a loop we have for - each loop

ex:- For (int i : a)

→ It work differently for each loop as if we write For (int i : a) it means the i value is not

__/__/____

not pointing to the index of vector but it will point the value at that index.

```
# include <iostream>
# include <vector>
  using namespace std;
   int main ()
{ vector <char> cars = {"wagnor", "mahindra", "kia"};
  for (char i : cars)
  {
    Cout << cars << end :;
  }
   return 0;
```

vector Functions

1) size
2) push_back () → to add value to the vector whose size not defined
3) capacity ()
4) at ()
5) Front ()
6) back ()
7) POP_back ()

→ for inserting the vector values

```
vector <int> vec;
 vec . push_back (25);
 vec . push - back (40);
 vec . push_back (20);
  vec . size ();
   Cout << vec . at [3];
```

136 - Leetcode

__/__/____

```
vec.pop_back()
cout << vec.front();
```

Memory
- Heap
- static
- dynamic

Q1 # find out unique number in an array

0	1	2	3	4
4	1	2	1	2

XOR
0 ^ 0 = 0
1 ^ 0 = 1
0 ^ 1 = 1
1 ^ 1 = 0

```
0100
0001
----
0101
0010
----
0111
0001
----
0110
0010
----
0100
```

program:-
```
int main()
{ int value, unq; vector <int> vec;
  cout << "Enter integers (-1 to stop)";
  while (cin >> value && value != -1)
  {
    vec.push_back(value);
  }
  unq = find_unique (vec);
  return (0);
}
```

* q. using vector linear search
*ACQ. print vector in reverse

__/__/____

```
int Find-unique (vector <int> vec)
{   int ans = 0;
    for (int i : vec)
  {   ans = ans ^ i;
  }
   return (ans);
  }
```

Time complexity

→ The time complexity is not the actual execution time of the program.

→ The TC is known as to evaluate and compare different algorithms instead of looking at the actual run time for an algorithm

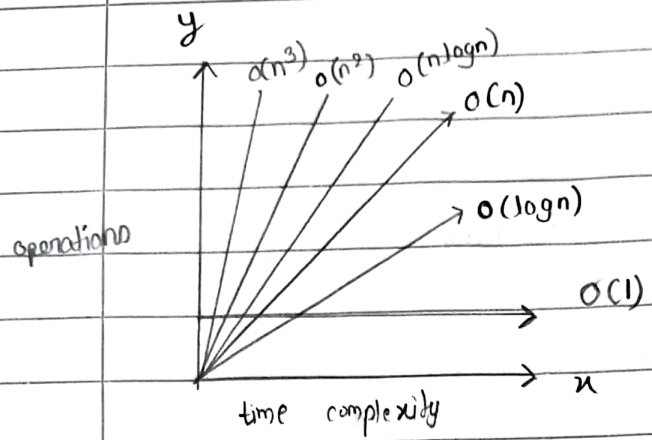
→ The TC is the number of operations needed to run an algorithm on large amount of data

→ Number of operations can be considered as a time because the computer uses sometime for each operation

→ • Best case (omega) → is always '1'. [sorted data]
• worst case (O 'Big O') → when data is very large
• average case (Θ) → [we are getting result in somewhere bt^n data]

→ linear search algorithm worst case TC is order of n because the searching number can be found at the n^th location in an array

→ linear search best case TC is order of 1 because the searching element can be found at location / index 0.



Hashtable

→ Hashtable is a data structure which is used to access faster data which stores in an associative manner it means the key and value pair of the data.

→ Hashtable uses an array data structure as a storage medium and uses hash technique to generate an index where an element is to be inserted

→ The collision resolution technique in the hashtable :-

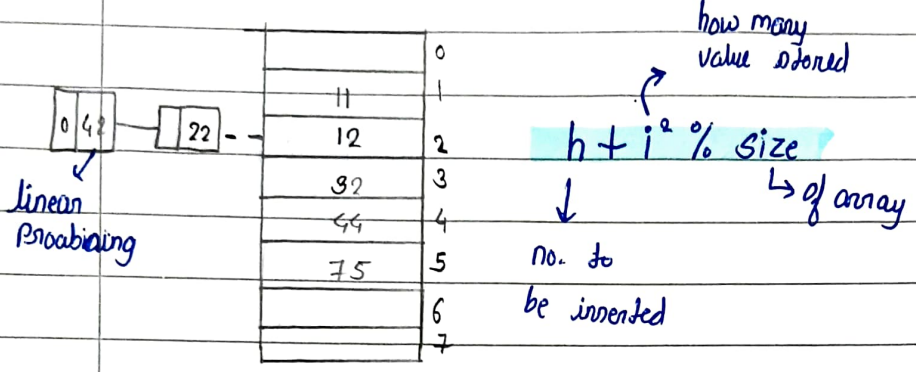
Collision resolution technique

- Collision :- When two values are getting stored in same index value. / when the hash fun" generates same index for multiple keys there will be a conflict i.e which value to be stored in that index
- chaining :- Known as ~~openin~~ open hashing. In this case we create a linklist node to point out the same index location to store the given value. chain is created outside of the hash table therefore it is known as ~~openi~~ open hashing.

4 close hashing :-

__/__/____

i) Linear probing
2) quadratic probing
3) Double ~~hash~~ hashing



Q.1 vector :- find ~~unique~~ vector linear search

```
# include <vector>
# include <algorithm>
# include <iostream>

int main()
{ std :: vector <int> myVector = {10, 20, 30, 40, 50};
  int targetValue = 40;
  auto it = std :: find (myvector.begin(),
        myvector.end(), targetvalue);
  if (it != myvector.end())
{  int index = std :: distance (myvector.begin(), it);
std :: cout << "element " << targetvalue << " found at index"
<< inden << std :: end l; }
  else { std :: cout << "element " << target value
  << "not found." << std :: end ; }
  return 0; }
```

__/__/____

Assignment 1 :- Movie cinema hall ticket booking using array?

→ syntax:- Count, delete.
do, prize

★ Q.1 The keys 1, 3, 12, 4, 25, 6, 18, 20, 8 are inserted into the hash table of length 10 using open hashing with the hash fun^n $h(i) = i^2 \bmod 10$ what is resultant table and find out the maximum prob value

$h(1) = 1^2 \% 10 = 1$
$h(3) = 9 \% 10 = 9$
$h(12) = 144 \% 10 = 4$
$h(4) = 16 \% 10 = 6$
$h(25) = 625 \% 10 = 5$
$h(6) = 36 \% 10 = 6$ ---probing
$h(18) = 324 \% 10 = 4$
$h(20) = 400 \% 10 = 0$
$h(8) = 64 \% 10 = 4$

0	20
1	1
2	8
3	
4	12
5	25
6	4
7	6
8	18
9	3

probing :- insert value at any next empty space
∵ for 8 index 4 come but 4 was already fill ∴ we fill it in next empty space (immediate one) which makes 8 number with maximum probing i.e 9

__/__/____

- Quadratic probaing

$h(K) = (h(K) + i^2) \% \text{ size}$

$h(K) = h(K) \% \text{ size}$

Q.1 Key :- 42, 16, 19, 33, 18, 27, 36, 62

$h(42) = 42 \% 10 = 2$

$h(16) = 16 \% 10 = 6$

$h(19) = \quad = 9$

$h(33) = \quad = 3$

$h(18) = \quad = 8$

$h(27) = \quad = 7$

$h(36) = \quad = 6 \rightarrow 1$

$h(62) = \quad = 2$

disadvantage :- no guarantee to find missing slot.

$h(36) = (h(K) + i^2) \% \text{ size}$

$h(36) + 4^2 \% 10$

$36 + 16 \% 10$

$52 \% 10$

2

$36 + 5^2$

$36 + 25 = 51 \% 10$

$= 1$

@ here i = index not available [prob] after how many boxes we can find empty space

__/__/____

→ In quadratic probing we need to find out the proab value and the sq. the prob (empty space) add it into the key and again mod by table size, the generated key will be the index location for the hash key to store the data.

- Double hashing

$h_1(K)$ = K % size

$h_2(K)$ = 8 − (K mod size)

Q.1 20, 34, 45, 70, 56 size = 11

0	
1	34
2	
3	
4	70
5	
6	
7	45
8	
9	20
10	
11	

20 % 11 = 9

34 % 11 = 1

45 % 11 = 1 → 2nd has since collison :-

8 − (45 % 11)

8 − 1

70 % 11 = 1

56 % 11 = 1

(8 − 1) = 7

@ not gurantee that index value will be found.

Q. Reverse the number :-

```
# include <iostream>
# include <vector>
using namespace std;
int main ()
```

__/__/____

Recursion

→ The process in which a function calls itself directly or indirectly is called recursion.

1)

Indirect

```
void add()
{ sub();
}
 void sub()
{ add();
}
```

Direct

```
void add()
{
   add();
}
```

To implement recursion we need to follow the steps :-

i) define a base case [This is the stopping cond" for recursion it prevents the function from infinitely calling itself.]

ii) Define recursive case it means problem is divided into smaller sub-problems

iii) ~~They~~ ensure recursion terminates properly.

iv) The time complexity of recursive fun" is order of 'n'

v) In recursion memory uses a stacked ds

Fun" Factorial n!

②

```
int main();
{
   cout << " factorial = " << fact(5);
   return 0;
}
```

__/__/____

① int fact (int n);
{
if (n == 1) → (base case)
{ return 1;
} else {
return (n * fact(n-1)); → (recursive case);
}

return (1)
2* Fact (1) 2 * ① = 2
3* Fact (2) 3* 2 = 6
4* Fact (3) 4* 6 = 24
5* fact(4) 5* 24 = 120
main ()

Prog:- ②

```
#include <iostream>
using namespace std;
void display (int n);
int main ()
{
  display (3);
  return 0;
}
void display (int n)
{ if (n < 1)
{ return ; → returning ctrl ↗ where the fun^n was called
}
else
{
  cout << n << "\t"; → (3 2 1 1 2 3)
  display (n-1);
  cout << n << "\t";
}
}
```

going back to place from

prog:- ③ sum of digit :- return (num % 10 + sum (num / 10));

```
int main()
{ sum (4321)
    return 0;
}
  int sum (int n)
  {
    if (n==0)
  { return 0;
  }
  return ( num % 10 + sum (num /10));
  }
```

UNIT 2 : SEARCHING AND SORTING TECHNIGUE

prog ④ :- Fibonnaci

Fibonnaci search

→ Fibonnaci Search is compeare based system that uses fibonnaci number to search an element in a sorted array

→ similarities of FS and binary search are both works on sorted array. It uses divide and conquer strategy

→ The time complexity of FS & BS is logn

→ differences of FS and BS is FS divide given array into unequal parts ; BS uses division operator to divide the range where FS uses Plus & minus operators. FS is examine as relatively closer elements in subsequent steps when the input array is weak and cannot fit in CPU cache memory in

__/__/____

that case FS is useful.

↳ ALGORITMIC STEPS

Step 1) Find the immediate FS number that is greater than or equal to >= the size of input array,

2) Then ~~fold~~ hold 2 preceddling number of FS

Step 2) When we need to find a key element in a left-hand side no need to compute the offset value by default offset value is '-1', offset makes the position of the last eliminated ranges from starting point

Step 3) When we need to find a key element on a right hand side of index i we need to move back F(K-1) of FS number and need to compute offset value as i = min (offset + a, n-1)

★ Ex:- int Fib-search (int arr [], int n, int val)

{

```
int a=0, b=1, c=1;
while (c <n)
{ a=b;
  b=c;
  c = a+b;
}
int offset = -1
while ( c >1)
{ int i= min (offset +a,
      n-1) ; }
if (arr[i] <val)
 {
   c=b ; c=3
   b=a ; b=2
   a= c-b ; a=1
   offset = i ; =4 }
else if (arr [i] >val)
{ c=a ;
  b=b-a ;
  a= c-b ;
}
else
```

```
return i;
if (b && arr[offset+1] == x)
return (offset+1);
   return -1;
```

Val

12	35	46	53	63	78	90	97
0	1	1	2	3	5	8	13

(1<8)	(2<8)	(3<8)	(5<8)
a=1	a=1	a=2	a=3
b=1	b=2	b=3	b=5
c=2	c=3	c=5	c=8

```
int min(int a, int b)
{
    return ((a>b)?b:a);
}
```

SENTINAL SEARCH

```
int sent_search (int arr[], int n, int key)
{
  int last;
  last = arr[n-1];      last =75
  arr[n-1] =key;
  while (arr[i] != key)
  {
      i++;
  } arr[n-1]=last;
  if (i<n-1 || arr[n-1] == x)
        return i;
   else
        return -1;
  }
```

sentinal search is a specific linear search, it is a variation of disadvantage of linear search is if we want to search a number not present

__/__/____

in array in this case the iterations will be n+1 because we are checking the for loop condition whether it is true or false in this case the iteration will goes beyond the out of bound of an array so the theoratical time complexity in the worst case is order of 2n+1 so the number of iterations for the above example is ~~15~~ 15, so to overcome this disadvantage of linear search we use a sentinal search in which the time complexity will be order of 'n' only in a worst case it means the no. of iterations = length of array

Q.1 Write a program or function to store role numbers of student in an array who attended training program in random order. write a searching function whether particular student attended training program or not. using sentinal search

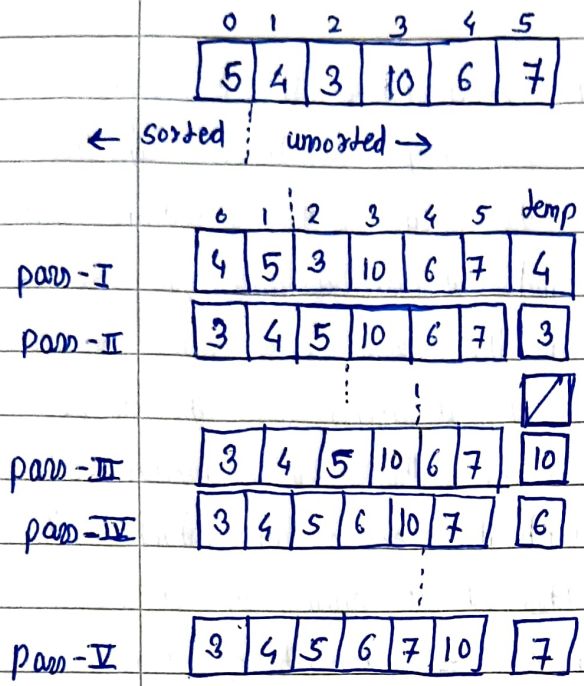
Sorting techniques

o Insertion Sort [time complexity = n^2]

→ The given list is divided into two sub-list 1. sorted sublist 2. unsorted sub-list

→ The 1st value [0] is consider as a sorted sublist and will check value from unsorted sub-list and placed it at a proper location of a sorted sub-list

__/__/____



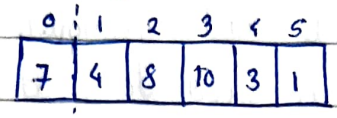

```
void insertionsort (int a[ ], int n)
  for (i=1; i<n ; i++)
  { temp = a[i] ;
    j = i-1 ;
  while (j>=0 && a[j]>temp)
  { a[j+1] = a[j];
    j--;
  }
  a[j+1] = temp ;
  }
}
```

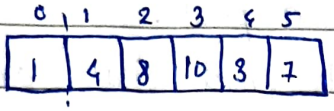
```
int main ()
{ int arr[7] ; int i, int j, int temp;
  int n;
  cout << " enter array << arr ;
  cin <<arr;
  insertion sort ( arr[ ], n)
  cout <<" enter size";
  cin >> n;
  for ( i=0 ; i<n ;i++)
  {
    cin >> a[i]

  cout << " Sorted element :" <<endl;
  for(i=0 ; i<n ; i++)
  { cout << a[i] << " ";
  }
}
```

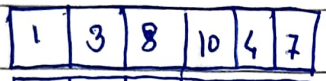
we cannot write a[7] a[n] since we have not took 'n' value = compile error
∴ a[10]
or # define n 10;
a[n]

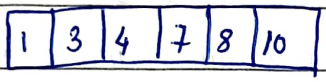
- Selection sort [time complexity n^2] void selectionsort (int a[], int n)

↳ ascending :-










```
for (i=0; i<n-1; i++)
{ int min = i;
  for (j = i+1; j<n; j++)
  {
    if (a[j] < a[min])
    {
      min = j;
    }
  }
  temp = a[min]
  a[min] = a[i];
  a[i] = temp;
}
```

→ whoever the size of array is small :-

```
int main ()
{
```

- Best case TC :- $O(n^2)$
- worst case TC : $O(n^2)$

__/__/____

- quick sort algorithm

→ QS algorithm works as divide and conquere strategy in a quick sort algorithm we need to select a key element which is known as a pivot.

→ The pivot can be the first, mid, last or any element of an array

→ time complexity is O (n log n)

steps :-

1. select pivot element assign a value to variables i and j as i = starting index of an array and j = the last index of an array (n-1)
2. if i < j we need to check a[i] =< a[pivot] the increment i index
3. if a[i] is not less that a[pivot] stop the i (++) increment and check a[j] > a[pivot] from the last index of an array then (--) decrement the a index
4. Whereven j -- stops we need to check if i < j then swap the values of index location i & j
5. whenever i crosses the j index location we need to swap pivot value with j index location
6. whereven we swap pivot with j in these is the values of LHS of pivot will be less than pivot and RHS values will be > than pivot
7. After swapping the pivot the given array will be divided into 2 partitions and eg again we need to call a recursive quick sort with these partitions

* [Passes are compulsory]

__/__/____

```
void quicksort (int a[], int first, int last)
{                    *      index
  if (first < last)
  {
    pivot = first;
    i = first;
    j = last;
    while (a[i] <= a[pivot] && i < last)
    {
      i++;
    }
    while (a[j] > a[pivot])
    { j--;
    }
    if (i<j)
    { temp = a[j];
      a[j] = a[i];
      a[i] = temp;
    }
}
```

```
while i<j
temp = a[pivot];
while a[pivot] = a[j]
  a[j] = temp;
  quicksort (a, first, j-1)
  quicksort (a, j+1, last) }
```

ex:-

0	1	2	3	4	5	6
23	15	45	18	32	7	60

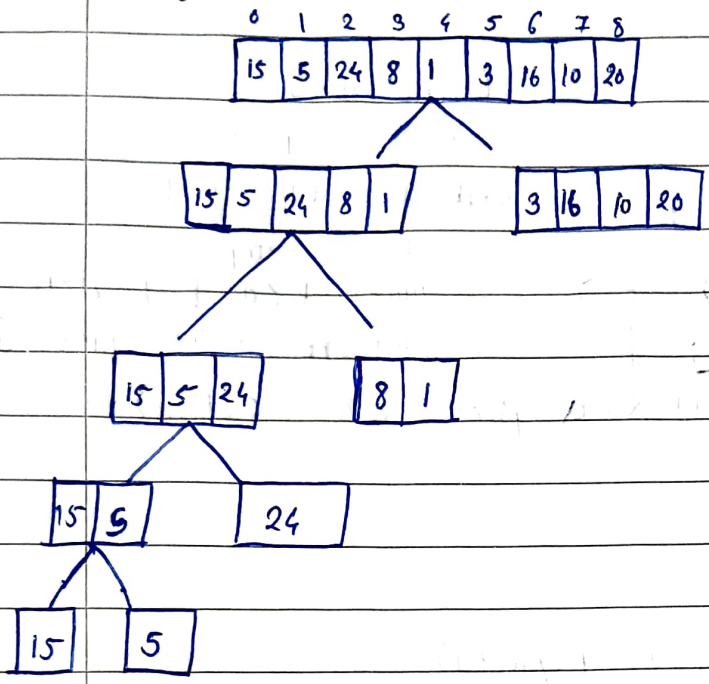
23	15	7	18	32	45	60

18	15	7	23	32	45	60

0	1	2
18	15	7

__/__/____

- Merge Sort [O = nlogn]



lower bound = 0

upper bound = n-1

mid = (lb+ub)/2 = 4

```
mergesort (A, lb, ub)
{
  if (lb < ub)
  {
    mid = (lb+ub)/2;
    mergesort (A, lb, mid);
    mergesort (A, mid, ub);
    merge (A, lb, mid, ub);
  }
}
```

```
void merge (A, lb, mid, ub)
{                (mid+1)
  i = lb, j = ub, k = lb;
while (i <= mid && j <= ub)
  {
    if (a[i] < a[j])
    {
      a[k] = a[i];
      k++; j++;
    }
    else
    {
      b[k] = a[j]
```

```
        K++ ; j++ ;
      }
  }
  while (i <= mid)
  {
    b[k] = a[i];
  }
  while (j <= ub)
  {
    b[k] = a[j];
  }
}
for (i = lb; i < ub ; i++)
{
  a[i] = b[i];
}
}
```

Q. 1 Delete repetative elements from array

```
# include <iostream>
using namespace std ;
int main ()
{
  int arr [] = {1, 2, 3, 3, 4, 5, 5} ;
  int n = size of (arr) / sizeof (arr [0]) ;
  for (int i = 0 ; i < n ; i++)
    { for (int j = i + 1 ; j < n;) }
```

```
{ if (arr[i] == arr[j])
{ for (int k= j; k<n-1 ; k++)
    { arr[k] = arr[k+1];
    }
  n--;
}
else
{
  j++;
}
}
}  cout << "sorted array :"
for (int i=0 ; i< n ; i++)
  { cout << arr[i] << " ";
  }
  cout return 0;
}
```

Radix sort

→ In a radix sort technique we sort the data digit by digit starts from LSB to MSB [most significant bead]

→ Radix sort is not comparison based sorting technique in this technique we need to find the maximum no. and count the number of digit of the maximum number

→ Make all the numbers equal to the maximum numbers digit count. [add zeros to LHS]

→ For sorting we need to create a bucket which consider as the base of a number which is in a

__/__/____

range 0 to 9

g.1 1, 10, 802, 2, 103, 11, 15, 321, 123, 90

↳ max. no [3-digit]

001, 010, 802, 002, 103, 011, 015, 321, 123, 090

pass I:
0 : 010, 090
1 : 001, 011, 321
2 : 802, 002
3 : 103, 123
4 :
5 : 015
6 :
7 :
8 :
9 :

pass II:
0 : 001, 802, 002, 103,
1 : 010, 011, 015
2 : 321, 123
3 :
4 :
5 :
6 :
7 :
8 :
9 : 090

pass III
0 : 001, 002, 010, 011, 015, 090
1 : 103, 123
2 :
3 : 321
4 :
5 :
6 :
7 :
8 : 802
9 :

pass I: 010, 090, 001, 011, 321, 802, 002, 103, 123, 015

pass II : 001, 802, 002, 103, 010, 011, 015, 321, 123, 090

pass III :- 001, 002, 010, 011, 015, 090, 103, 123, 321, 802

ex:-

	0	1	2	3	4	5	6	7	8	9	10	11	
a =	2	1	1	3	0	4	5	2	3	4	5	1	n = 12

; K = 5 (biggest NO in the array)

★ count array will be the index for the array of 'a' values.
count array & index will be refered as value of array 'a'

count =	0	1	2	3	4	5
	1	3	2	2	2	2

→ value of array a

↳ count of individual no. of array 'a'

T.C = [O = n+a]

__/__/____

```
for (i=0 ; i<n ; i++)
 {
    count [a[i]]++ ;
 }
```

★ update the count array from index location '1' by adding previous index + the next loc. index value

count =

0	1	2	3	4	5
1	4	6	8	10	12

Sorted =

0	1	2	3	4	5	6	7	8	9	10	11
0	1	1	1	2	2	3	3	4	4	5	5

★ To sort an array check the value right to left as index of II because to maintain the stability of counting sort algorithm

```
for (i=0 ; i<n ; i++)
 {
    count [a[i]] ++;
 }
 for (i=1 ; i<=k ; i++)
 {
   count [i] = count[i] + count [i-1] ;
 }
 for (i=n-1 ; i>=0 ; i--)
 {
   b[ -- count [a[i]] = a[i] ;
 }
```

__/__/____

Q.1 Define ~~extract~~ abstract data type list the linear and non-linear data structure?

Q.→ 2 Differentiate between iterative and recursive algorithm with an example [program of factorial

Q.3 Demonstrate the use of open addressing a collision handling with an example. [Hash table]

Q.4 Compare open addressing and chaining method in terms of memory usage and efficiency?

Q.5 Explain mid square and digit folding

Q.6 sort the given array using insertion sort

Q.7 Explain and demonstrate sorts [pseudo code & passes]

Q.8 Explain fibonacci search algorithm and analyse the time complexity.

Q.9

__/__/____

ex:- 432, 8, 530, 90, 88, 231, 011, 045, 677, 199

→ Step 1:- find the maximum number and count the digits of max no. make all other numbers equivalent to the maximum number digits

Step 2: we use counting sort technique with the base value of number

Step 3: Draw the count array, apply the count sort for ones place of each number

bucket / count array :-

0	1	2	3	4	5	6	7	8	9
2	2	1	0	0	1	0	1	2	1

updated count array :-

0	1	2	3	4	5	6	7	8	9
2	4	5	5	5	6	6	7	9	10-1

b =

0	1	2	3	4	5	6	7	8	9
530	090	231	011	432	045	677	008	088	199

See numbers from array from right to left ; then see ones value (place) see in updated count array index and -1 the value present in that index whatever it makes put the that number on that index in 'b' array

pass II :-

0	1	2	3	4	5	6	7	8	9
1	1	0	3	1	0	0	1	1	2

0	1	2	3	4	5	6	7	8	9
1	2	2	5	6	6	6	7	8	10

c =

0	1	2	3	4	5	6	7	8	9
8	011	530	432	231	045	677	88	90	199

__/__/____

pass III :-

	0	1	2	3	4	5	6	7	8	9
	5	1	1	0	1	1	1	0	0	0

	0	1	2	3	4	5	6	7	8	9
234	5	6	7	7	8	9	10	10	10	10

	0	1	2	3	4	5	6	7	8	9
d =	8	11	45	88	90	199	231	432	530	677

```
Radix sort (A,n)
{
   max = getmax (A,n);
   for (pos=1; max/pos>0 ; pos*10)
   {
      countsort ( A, n, pos);
    }
```

- Counting sort (A,n,pos)

```
{
   int  count [10] = {0};
    for (i = 0; i<n ; i++)
    {
      count [(a[i] /pas) /10]++;
    }
    For (i=1 ; i<=k ; i++)
    {
      cout Count [i] = count [i] + count [i-1];
    }
```

```
for(i= i-1 ; i >=0; i--).
{
b[-- cout[(a[i] /pos) % 10] = a[i];
}
for (i=0 ; i<n ; i++)
{
   a[i] =b[i];
}
}
```

UNIT 3 : LINKED LIST

→ Heap memory

→ user define function :- class, construction, structure

→ It stores at a time two variable i.e [datatype & address]

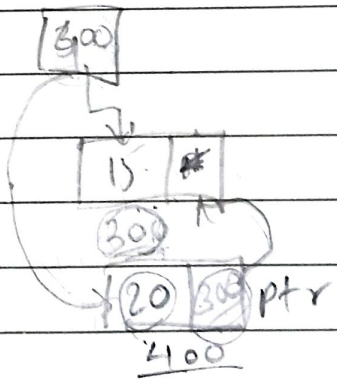

```
Struct Node
{
   int data;
   struct node *next;
};
struct Node * head = NULL;
```

RLE ★ Music player / on any ~~bad~~ thing that contain queue or list.

__/__/____

Insert at beginning

```
void insert-begin ()
{
   struct Node *ptr = new Node ();   // Node → datatype
   if (ptr == NULL)
   {
      cout << " memory is not allocated " <<endl;
   }
   else
    {
       cout << " enter value " <<endl;
       cin >> val;
       ptr -> data = val;
       if (head == NULL)
       {
         ptr -> next = NULL;   ptr -> data = val;
            head = ptr;
         }
    else
    {
       ptr -> data = val;
       ptr -> next = head;
        head = ptr
    }
```



- Time complexity O(1)

insert at last

__/__/____

→ Time complexity O(n)

```
void insert_last ()
{
  struct node *ptr, *temp ;
  ptr = new Node ();
  if ( ptr == Null)
  {
   cout << "memory is not allocated";
  }
  else
   {
     cout << " enter value ";
     cin >> val;
   if (head == Null)
   { ptr -> data = val;
     ptr -> next = Null ;
                ptr ;
   }
  else
   {
     ptr -> data = val
     temp = head
    while (temp -> next != NULL)
     {
       temp = temp -> next ;
     }
     temp -> next = ptr
     ptr -> next = NULL ;
     cout << " Node is inserted";
     }
   }
}
```

__/__/____

```
void insert_position ()
{
  int i, loc, val;
  struct Node *ptr, *temp;
    Ptr = new Node();
    if (Ptr == NULL)
    {
  cout<<"Memory not allocated"
    }
    else
    {
     cout << "Enter value : " <<endl;
      cin >> value;
     cout << "Enter location : " <<endl;
      cin >> loc;
    }
    For (i=0 ; i< loc ; i++)
    {
      ~~temp~~ temp = temp → next;
      if (temp == NULL)
      {
        cout << " can't insert ";
      }
    }
    ptr → data = val;
    ptr → next = temp → next
    temp → next = ptr;
```

" for insertion at the location of node in a link-list we need to write the cond^n @ i= for (i=0; i< loc-1 ; i++)

ii. If we want to add a ~~no~~ node after the location we write down the cond^n as i < loc

iii. If we want to add a new node before the loc we need to write the cond^n as i < loc - 2

__/__/____

Delete - begin

```
void delete_begin ()
{
    struct Node *ptr;
    if (head == NULL)
    {
        cout << "empty";
    }
    else
    {
        cout << "~~is head of data~~ head -> data << " is deleted "
        << endl;
        head = head -> next;
        free (ptr);
    }
}
```

Delete -> last

```
void delete_last ()
{
    struct Node *temp;
    if (head == NULL)
    {
        Linked list empty;
    }
    else
```

```
{ temp = head;
  if (head -> next == NULL)
{
   cout << head -> data << "is defined" << endl;
    head = NULL
    free (temp);
 }
else
 {
   while (temp -> next != NULL)
      {
        temp = temp -> next ;
      }
     cout << temp -> data << " is detected" ;
      temp -> next = NULL ;
      free (temp);
    }
 }
```

Delete position

```
void delete_position ()
{
   struct Node * temp *temp1;
    int loc , i,
    cout << "enter location :" << endl;
    cin >> loc;
    temp = head;
    for (i = 0; i < loc ; i++)
```

```
        temp1 = temp ;
        temp = temp -> next ;
    }
    temp1 -> next = temp -> next ;
    temp -> next == NULL ;
     cout << temp -> data << " is deleted" ;
     free (temp) ;
}
```

Search number

```
void search ()
{
   struct Node * temp ;
   int flag=0 , val ;
    cout << " enter number you want to search " ;
     cin >> val;
     temp = head ;
     if (head == ~~NULL~~ NULL)
    {
      Linked list is empty ;
    }
    else
    {
      while (temp != NULL)
      {
    ~~if temp = temp -> Next ;~~
        (if temp -> data == val)
```

```
{
        flag = 1;
}
else
{
  temp = temp->next;
}
}
if (flag == 0)
{
  cout << " No. is not found ";
}
else
{
  cout << " No is found";
}
}
```

Reverse

```
void reverse()
{ struct Node * Reverse (struct Node * head)
    struct Node * curr = head, * prev = NULL, * nextnode;
  while (curr != NULL)
  {
    nextnode = curr -> next;
    curr -> next = prev;
    prev = curr;
    curr = nextnode;
  } return prev; }
```

```
struct Node * insert_sort (struct Node *head)
{
  struct Node *ptr, *temp ;
  int val ;
  ptr = new Node ();
  if (ptr == NULL)
  {
    cout << "memory is not allocated" << endl;
  }
  else
  {
   cout << "enter value : " << endl;
    cin >> val ;
   temp = head;
   if (head == NULL)
   {
    ptr -> data = val;
    ptr -> next = NULL ;
    head = ptr;
  }
  else if
       (head -> data > val)
   {  ptr -> data = val;
      ptr -> next = head;
      head = ptr;
   }
  else
  {
    while (temp -> next != NULL && ~~&&~~
           temp -> next -> data < val)
```

__/__/____

```
    temp = temp -> next;
    }
    ptr -> data = val;
    ptr -> next -> temp -> next;
    temp -> next = ptr;
    }
}
return (head);
}
```

** Q.1 A riding app (Ola / Uber) maintains driver and passenger information as a developer you are asked a riding app (Ola, Uber) maintain as a developer you are asked to design abstract datatype, a trip entity define the add and show how it can store and manage trip details ex: pickup / drop / price / driver's info

→

```
class trip  [abstract datatype for trip entity]
{
  public :
      pickup : string
      drop : string
      price : float
      drivers ID : int / string
      create ()
      update fare ()
    }
```

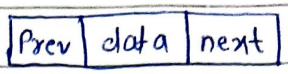
□ □ → →

__/__/____

Void Insert_position() [using 2 ptr pointers]
{
struct Node *ptr, *temp, *temp1;
ptr = new Node();
if (ptr == NULL)
{
memory is not
}
else
{
cout << "enter value";
cin >> val;
cout << "Enter ~~val~~ pos: ";
cin >> ~~no~~ ~~pos~~ loc;
temp = head;
for (i=0; i<loc; i++)
{
temp1 = temp;
temp = temp -> next;
if (temp == NULL)
{
cout<<" cannot insert";
}
}
ptr -> data = val;
temp != next = ptr;
ptr -> next = temp;

__/__/____

Doubly linked list



Syntax:-

```
struct Node
{
  int data;
  struct Node * prev , * next ;
};
```

```
Void Insert_begin ()
{
  struct Node * ptr ;
   ptr = new Node ()
   if (ptr == NULL)
   {
   "memory is not allocated"
   }
   else
   {
     cout << "Enter value :";
     cin >> val ;
     if (head == NULL)
     {
       ptr → data = val;
       ptr → next = NULL;
       ptr → prev = NULL ;
       head = ptr;
     }
```

```
else
{ ptr → data = val;
  ptr → prev = NULL;
  ptr → next = head;
  head = ptr;
}
}
cout << " Node is inserted";
}
```

** Q.2 A social networking platform needs to store user connections explain with justification what you will choose linear or non-linear ds to represent friends and mutual friends with one example

Q.3 E-commerce platform sort the product reviews with the help of algorithmic complexity i.e best case, average case & worst case

↳ void insert_last ()
{
```
Struct Node *ptr, *temp;
ptr = new Node ();
if (ptr == NULL)
{
  cout << " memory is not allocated";
}
else
```

```
{
    cout << " enter value :" ;
    cin >> val ;
    if (head == NULL)
    {
        ptr -> data = val;
        ptr -> prev = NULL;
        ptr -> next = NULL;
        head = ptr;
    }
    else
    {
        temp = head;
        while (temp -> next != NULL)
        {
            temp = temp -> next;
        }
        ptr -> data = val;
        ptr -> next = NULL;
        ptr -> prev = temp;
        temp -> next = ptr;
    }
    cout << " node is inserted";
}
```

* (we cannot take ptr -> prev = head because then it will always add node immediate after head)

```
Void insert_position ()
{
    struct node *ptr, *temp;
    ptr = new Node ();
    if (ptr == NULL)
    {
        "memory is not allocated";
    }
    else
    {
        temp = head;
        cout << " Enter value & pos : ";
        cin >> val >> pos;
        for ( i=0; i<pos ; i++)
        { temp1 = temp ;
          temp = temp ->next;
        } if ( temp ==0)
        {
            cout << " cannot inserted";
        }
    } else
    {
        ptr -> data = val;
        ptr -> prev = temp1;
        ptr -> next = temp ;
        temp1 -> next = ptr ;
        temp -> prev = ptr ;
    }
}
```